Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 5

What you get . . . 5

What this delivers . . . 6

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 6

What this solution is . . . 6

Why this matters . . . 6

The data types in scope . . . 7

The tooling pattern (tool-agnostic) . . . 7

Coexistence and mail routing during the transition . . . 8

The cutover (DNS/MX) . . . 8

The named artifacts . . . 8

What this is not . . . 9

The deployment model . . . 9

2. Requirements Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 9

2.1 Source Exchange Online / M365 access for migration . . . 9

2.2 Google Workspace target tenant and licensing (super admin) . . . 10

2.3 A tooling & data-types design . . . 10

2.4 DNS / mail-flow control and a coexistence design . . . 10

2.5 A migration tool, configured . . . 11

2.6 A pilot batch and per-user validation . . . 11

2.7 A DNS/MX cutover runbook with rollback and change control . . . 11

Checklist . . . 11

Global Requirements — Google Workspace & Gemini Solutions 12

1. Google Workspace / Cloud Identity tenant . . . 12

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 12

3. DNS control for the routed domains . . . 12

4. Identity & provisioning source of truth . . . 13

5. A pilot scope (OU / group) and change control . . . 13

6. Security, compliance & governance stakeholders . . . 13

3. Implementation Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 14

Step 1 — Assess: baseline the source mailboxes . . . 14

Step 2 — Configure: fix the tooling and data-types design . . . 14

Step 3 — Configure: design and stand up coexistence . . . 15

Step 4 — Configure: build the pilot batch . . . 15
Step 5 — Author the DNS/MX cutover runbook and rollback . . . 16
Step 6 — Pilot: run Wave 0 end to end . . . 16
Step 7 — Validate: fix the runbook, not just the batch . . . 16
Step 8 — Expand wave by wave . . . 16
Step 9 — Decommission coexistence . . . 17
5. Testing Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 17
Test 1 — Batch membership is complete and correct . . . 17
Test 2 — Readiness: target accounts provisioned and licensed . . . 17
Test 3 — Mail migrated . . . 17
Test 4 — Calendar migrated . . . 18
Test 5 — Contacts migrated . . . 18
Test 6 — Coexistence routes mail correctly . . . 18
Test 7 — Cross-platform free/busy resolves . . . 18
Test 8 — DNS/MX cutover applied under change control . . . 18
Test 9 — Coverage and exceptions reported . . . 18
Test 10 — Rollback is proven available . . . 18
Pass/fail gate . . . 18
6. Troubleshooting Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 19
A. A user's mail migrated but calendar or contacts did not . . . 19
B. Migrated mail appears duplicated . . . 20
C. Mail is lost or double-delivered during coexistence . . . 20
D. Cross-platform free/busy fails . . . 20
E. The DNS/MX cutover did not take effect (or cannot be reversed quickly) . . . 20
F. A batch fails validation . . . 21
G. The same problem recurs every wave . . . 21
H. A cutover needs to be reversed . . . 21
I. Graph scripts fail to connect . . . 21
J. Diagnostics quick reference . . . 21
K. When to escalate . . . 21
7. Best Practices — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 22
8. Deliverables Summary — Exchange Online → Gmail Migration (Mail, Calendar, Contacts) 23
Documentation . . . 23
Named artifacts . . . 24
PowerShell scripts . . . 24
The mail-migration playbook (outline) . . . 25

What the buyer receives 25
Outcome 25
Appendix A — PowerShell Scripts 26

Executive Summary — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Email is the highest-visibility, highest-risk part of a Google move — users feel every lost item, broken calendar, and mis-delivered message the moment it happens. Most mail migrations stumble the same way: no pilot, so an architectural problem surfaces at fleet scale instead of on 50 users; coexistence as an afterthought, so mail is lost or double-delivered while the estate is split; the DNS/MX cutover treated as a switch flip with no rollback; and no per-user validation, so a mailbox is called "migrated" while its calendar and contacts never arrived. This solution migrates **Exchange Online mailboxes to Gmail — mail, calendar, and contacts** — with the right tooling pattern, coexistence/mail-flow during the transition, a governed DNS/MX cutover, and per-user validation, executed **pilot-first** and **rollback-ready**.

This is a complete, run-it-the-repeatable-way package written for a migration lead and mail-flow owner. Every artifact is named once and reused across every wave, so the same proven runbook executes from the pilot batch through the last business unit. It is authored **tool-agnostically**: it governs *which pattern and when* — Google's built-in **data migration service** (Admin console, with delta migrations), the **desktop tool (GWMME)** for mail/calendar/contacts in one run, or a named third-party tool — while deferring each tool's internal screens to the vendor's own docs.

What you get

- A **Solution Overview** of the Exchange-to-Gmail model — the three data types (mail, calendar, contacts), the tooling patterns, split/dual-delivery coexistence, and the DNS/MX cutover — with the named artifacts.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite: source Exchange Online / Entra access, a licensed Google Workspace target, a frozen tooling & data-types design, DNS control and a coexistence design, the migration tool configured, a pilot batch with per-user validation, and a DNS/MX cutover runbook with rollback.
- An **Implementation Guide** of precise, ordered steps: baseline the mailboxes, design tooling and coexistence, build a named pilot batch, migrate mail/calendar/contacts, validate each user across all three data types, manage coexistence, cut over DNS/MX under change control, then expand wave by wave and decommission coexistence.
- **Five PowerShell scripts** — source-estate discovery, per-user migration validation, a coverage/exception report, safe remediation, and a wave target-group builder — read-only against Microsoft Graph except the two write scripts, which default to `-WhatIf`.
- **Testing and Troubleshooting** guides with explicit per-batch gates (mail, calendar, contacts, coexistence, cutover), plus Best Practices for cutover and coexistence.

This solution **references, rather than duplicates,** the M365 ↔ Google coexistence and the broader M365 → Google Workspace migration-program siblings: the program sequences and gates the waves across mail, Drive, and chat; this solution owns the mail, calendar, and contacts mechanics and the mail-flow cutover.

What this delivers

Every in-scope mailbox on Gmail with **calendars and contacts intact**, coexistence managed during the transition, the DNS/MX cutover governed and reversible, and the whole migration proven on a pilot batch before any production wave.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — with per-user validation across mail/calendar/contacts and a change-controlled, reversible DNS/MX cutover before each wave, and coexistence decommissioned only when the estate is fully on Gmail.

Outcome

Mailboxes move to Gmail cleanly — nothing lost, calendars and contacts intact, mail flow unbroken through coexistence, and the cutover proven on a pilot before it scales.

1. Solution Overview — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

What this solution is

Email is the highest-visibility, highest-risk part of a Google move: it is where users feel every delay, every lost item, and every mis-delivered message immediately. This solution migrates **Exchange Online mailboxes to Gmail** — **mail, calendar, and contacts** — with the right tooling pattern, a **coexistence / mail-routing** design for the period the estate is split, a governed **DNS/MX cutover**, and **per-user validation**, executed **pilot-first** and **rollback-ready** so nothing is lost and users land smoothly. It is the per-service mail-migration workstream that the broader **M365 → Google Workspace migration program** sequences and gates; where that program owns segmentation and program governance across mail, Drive, and chat, this solution owns the **mail, calendar, and contacts** mechanics and the mail-flow cutover for a given wave.

Why this matters

Mail migrations fail for predictable reasons. There is no pilot, so an architectural problem — a mis-mapped resource calendar, a broken free/busy lookup, a routing loop — is discovered at fleet scale instead of on 50 users. Coexistence is an afterthought, so while half the estate is on each platform, inbound mail is lost or double-delivered and cross-platform meetings fail. The DNS/MX cutover is treated as a flip of a switch rather than a change-controlled step with a back-out, so a bad change takes mail down with no rollback. And nobody validates per user, so a mailbox is declared "migrated" while its calendar events, contacts, or recent mail never arrived. A disciplined approach fixes the cause: it picks the tooling deliberately, designs coexistence up front, proves the whole cutover on a named pilot batch, validates mail **and** calendar **and** contacts per user, and only then cuts over DNS/MX and expands by wave. Google's own guidance is consistent — migrate email, calendar, and contacts through a supported tool, run **delta migrations** so re-runs bring only new mail without duplicating, and route mail during the transition with **split or dual delivery** ([Google — migrate email with the data migration service](https://knowledge.workspace.google.c

om/admin/migrate/migrate-email-with-the-new-data-migration-service), [Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery), [Google — dual delivery](https://support.google.com/a/answer/9228551)).

The data types in scope

The migration moves three user-facing data types, each with its own validation:

• **Mail** — the mailbox contents (folders/labels, read/unread state, and history), migrated to

Gmail. Supported tools run **delta migrations**, so a second pass brings only mail that arrived since the last run without duplicating what already moved ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service)).

• **Calendar** — primary and additional calendars and their events (all-day events, attendees,

status, sensitivity, reminders), migrated to Google Calendar; **resource/room calendars** import when a suitable mapping is provided ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration)).

• **Contacts** — the user's contacts, migrated to Google Contacts. Contact migration is supported by

the desktop migration tool for Exchange (mail, calendar, **and** contacts in one operation) ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme)).

The tooling pattern (tool-agnostic)

This solution is authored **tool-agnostically** — it governs *which pattern and when*, not one vendor's console clicks. Two supported Google-native patterns exist, plus common third-party tools; choose per the estate and record the choice as `GWGMAIL Design - Tooling & Data Types - Mail`:

• **Data migration service (Admin console).** Google's built-in **data migration service** migrates

email from Exchange Online directly from the Admin console, with **delta migrations** and a per-batch user limit (Google states up to **250 users at a time**); calendar migration from Exchange Online is available alongside mail. Requires a Workspace **super admin** and, on the source side, a **Microsoft Entra administrator** to authorize the connection ([Google Workspace Updates — migrate email & calendar from Exchange Online](https://workspaceupdates.googleblog.com/2025/06/migrate-email-calendar-content-from-microsoft-exchange-online-to-google-workspace.html), [Google — use the default data import method for Exchange Online](https://knowledge.workspace.google.com/admin/migrate/migrate-data-from-an-exchange-online-account)).

• **Desktop migration tool for Exchange (GWMME).** The **Google Workspace Migration for Microsoft

Exchange **desktop tool migrates** mail, calendar, and contacts** for multiple users in one operation from Exchange/IMAP sources — the pattern to use when contacts must move as part of the same run or when the source is not covered by the Admin-console path ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme), [Google — what's migrated with GWMME](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-with-gwmme)).

• **Third-party migration tools.** Where an estate's scale, throughput, or data-shape needs it, a

third-party mail-migration tool is integrated at the pattern level. Its internal console steps are **deferred to the vendor's own docs** (per the Google-category vendor boundary); this solution governs the pilot-first sequencing, validation, and cutover around whichever tool is chosen.

> ***Vendor boundary.*** *Migration-**tool internals** (screen-by-screen operation of the data migration service job, GWMME, or a third-party tool) are deferred to the vendor's own documentation. This solution authors the **repeatable pattern** — assess, configure, pilot, validate mail/calendar/ contacts, manage coexistence, cut over DNS/MX, expand — around any of them.*

Coexistence and mail routing during the transition

While some users are on Gmail and some are still on Exchange Online, **inbound mail must reach the right platform for each user** and calendar free/busy should resolve across the divide. Two routing patterns cover this, per Google's docs:

- **Split delivery.** Point the domain's **MX at Google**; Gmail receives all inbound mail and

routes on to Exchange Online the messages for users who have not yet migrated, per a routing rule keyed to the migrated population. This is the usual pattern for a per-wave move ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery)).

- **Dual delivery.** Deliver a copy to a Gmail inbox **and** to the legacy system — used for

archiving or a phased overlap where both mailboxes receive mail ([Google — dual delivery](https://support.google.com/a/answer/9228551)).

Lower the domain's **MX TTL** (for example to 300 seconds) ahead of the cutover so routing changes take effect quickly and a back-out is fast. Coexistence is **transitional** — designed with an intended end and decommissioned once the estate is fully on Gmail, never left running indefinitely.

The cutover (DNS/MX)

The mail-flow cutover is a **change-controlled** step, not a switch flip. With MX pointed at Google and split-delivery routing carrying not-yet-migrated users onward, each wave's cutover moves that wave's users from "routed onward to Exchange" to "delivered in Gmail," and the final wave retires the onward route entirely. Every DNS/MX change is staged with a **documented, tested back-out** so a failed validation can be reversed under change control.

The named artifacts

Author these once and reuse the exact names across every wave, so the migration stays legible across owners and waves:

Artifact	Name
Source mailbox/estate baseline	`GWGMAIL Discovery - Source Mailboxes - Baseline`
Tooling & data-types design	`GWGMAIL Design - Tooling & Data Types - Mail`

Artifact	Name
Coexistence / mail-routing design	`GWGMAIL Coexistence - Mail Routing - Transition`
Pilot migration batch	`GWGMAIL Pilot - Mail Batch - Wave 0`
Per-wave target group	`GWGMAIL Wave - Target Group - Per Wave`
Per-user validation record	`GWGMAIL Validation - Mail/Cal/Contacts - Per User`
DNS/MX cutover runbook	`GWGMAIL Cutover - DNS/MX - Per Wave`

What this is not

This solution moves **mail, calendar, and contacts** and governs the mail-flow cutover. It does not own the program-level wave segmentation and cross-service governance (that is the **migration program** solution), it does not migrate Drive or chat (their own solutions), and it does not re-document a migration tool's internal screens (deferred to the vendor). Where the included scripts read the source estate to plan and validate, they read **Microsoft 365 / Microsoft Entra** through Microsoft Graph, **read-only** — they move no mail and change no mail flow.

The deployment model

> *Baseline the mailboxes and design the tooling/coexistence once, prove the full migration and cutover on a named pilot batch, validate mail **and** calendar **and** contacts per user against the gate, then cut over DNS/MX and expand wave by wave — the repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

2. Requirements Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Verify every requirement before running the migration. Each entry gives Description, Why, Verify, and Implement-if-missing. Category-wide prerequisites (the Google Workspace tenant, the source M365/Entra environment, DNS control, the identity source of truth, a pilot scope, and governance stakeholders) are in `global-requirements-google.md`; this guide adds what the Exchange-to-Gmail mail migration specifically needs.

2.1 Source Exchange Online / M365 access for migration

Description: Administrative access to the source **Microsoft 365 / Exchange Online** tenant sufficient to authorize a mailbox migration — the built-in data migration service requires a **Microsoft Entra administrator** on the source side to authorize the connection; the desktop tool (GWMME) requires the source mailbox access (service account / application access) its administration guide specifies. Why: The tool reads each mailbox's mail, calendar, and contacts from Exchange Online; without authorized source access no data can be read. Verify: You can authorize the data migration service against Exchange Online as a Microsoft Entra administrator, or GWMME can access the source mailboxes; run

`GWGMAIL-Discovery.ps1` to confirm you can read the source estate. Implement-if-missing: Obtain the Entra administrator role (data migration service) or configure the source mailbox/service-account access GWMME requires, per the tool's own documentation ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service), [Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme)).

2.2 Google Workspace target tenant and licensing (super admin)

Description: A provisioned Google Workspace tenant with a **super admin** account and Gmail-bearing licenses for the users being migrated, with target accounts provisioned (per the identity source of truth in the global requirements) before their mail lands. Why: Mail, calendar, and contacts land in a licensed Google account; the data migration service and GWMME both require a Workspace super admin, and a mailbox cannot migrate into an account that does not yet exist. Verify: Sign in to admin.google.com as super admin and confirm Gmail licenses for a sample wave; confirm those users are provisioned in the Admin console. Implement-if-missing: Provision licenses and the target accounts (GCDS/SCIM per the global requirements) before scheduling the pilot.

2.3 A tooling & data-types design

Description: A decision record — `GWGMAIL Design - Tooling & Data Types - Mail` — fixing the migration **tool/pattern** (data migration service, GWMME, or a named third-party tool), which **data types** move (mail, calendar, contacts) and in what order, resource/room-calendar and attendee **mappings**, and whether **delta migrations** will be used to finalize recent mail at cutover. Why: Every wave must run the *same* tooling and data-type decisions; deciding per wave guarantees rework and inconsistent outcomes. Delta migration is what lets a re-run bring only new mail without duplicating ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service)). Verify: The design names the tool, the data types and order, the required mappings, and the delta approach, and is agreed by the migration's technical owners. Implement-if-missing: Hold the design step (Implementation Step 2) and freeze the design before the pilot.

2.4 DNS / mail-flow control and a coexistence design

Description: Control of the domain's **DNS** (MX, SPF, DKIM, DMARC, TXT) and a coexistence design — `GWGMAIL Coexistence - Mail Routing - Transition` — defining **split delivery** (MX at Google, Gmail routes not-yet-migrated users onward to Exchange Online) or **dual delivery**, keyed to the migrated population, with a **lowered MX TTL** for a fast cutover and back-out. Why: While the estate is split, inbound mail must reach each user's current platform and free/busy should resolve across the divide; without a coexistence design mail is lost or double-delivered. Split delivery routes inbound mail to Google or Microsoft 365 per rule; dual delivery delivers to a primary and a secondary ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery), [Google — dual delivery](https://support.google.com/a/answer/9228551)). Verify: You can edit the authoritative DNS zone; the design specifies the routing pattern and the free/busy approach; a test message and a cross-platform meeting resolve correctly in a lab. Implement-if-missing: Coordinate DNS control, design coexistence in Step 3, stand up routing on a test host, and prove it before the first production wave. Coexistence is transitional — set its intended end.

2.5 A migration tool, configured

Description: The chosen migration tool is installed/authorized and reachable: the **data migration service** connection to Exchange Online is authorized, or **GWMME** is installed with source access per its administration guide, or the named third-party tool is licensed and configured. Why: The pilot cannot run until the tool can read the source and write to the target; tool configuration is a prerequisite, not a pilot step. Verify: A single test mailbox migrates a small set of mail/calendar/contacts to a target test account without error. Implement-if-missing: Configure the tool per its own documentation (deferred — vendor boundary), then prove one test mailbox before the pilot batch. For the data migration service, enable smart features/personalization **at least 24 hours before** the migration so migrated mail sorts and searches correctly ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme)).

2.6 A pilot batch and per-user validation

Description: A named pilot batch — `GWGMAIL Pilot - Mail Batch - Wave 0` — of a small, representative user set (core IT and early adopters), plus a per-user validation record `GWGMAIL Validation - Mail/Cal/Contacts - Per User` that confirms **all three** data types landed. Why: The full migration and cutover must be proven on a pilot before any production wave, and each migrated mailbox must be validated for mail **and** calendar **and** contacts — not declared done on mail alone. Verify: A named pilot batch exists and the per-user validation record covers the three data types; build the batch with `GWGMAIL-Set-ScopeTargeting.ps1` and validate with `GWGMAIL-Validation.ps1`. Implement-if-missing: Define the pilot batch, run it end to end (Implementation Steps 5–6), and validate per user before expanding.

2.7 A DNS/MX cutover runbook with rollback and change control

Description: A repeatable cutover runbook — `GWGMAIL Cutover - DNS/MX - Per Wave` — with the fixed sequence (readiness → migrate mail/cal/contacts → coexistence routing → validate → DNS/MX cutover → rollback) applied under **change control**, with a documented, tested back-out. Why: A DNS/MX change is high-impact; an ungoverned cutover with no rollback becomes an incident. Verify: The runbook exists with all stages and a documented rollback; the change-control path and the lowered MX TTL are in place. Implement-if-missing: Author the runbook in Step 5, prove it on the pilot, and reuse it unchanged for every wave.

Checklist

- Source Exchange Online / Entra access authorized — `GWGMAIL-Discovery.ps1`
- Google Workspace target licensed + accounts provisioned (super admin)
- Tooling & data-types design frozen — `GWGMAIL Design - Tooling & Data Types - Mail`
- DNS control + coexistence design proven — `GWGMAIL Coexistence - Mail Routing - Transition`
- Migration tool configured; one test mailbox proven
- Pilot batch defined + per-user validation — `GWGMAIL Pilot - Mail Batch - Wave 0`

• DNS/MX cutover runbook + rollback + change control — `GWGMAIL Cutover - DNS/MX - Per Wave`

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements

before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Follow these steps in order. This guide migrates **Exchange Online mailboxes to Gmail** — mail, calendar, and contacts — proving the full migration and DNS/MX cutover on a **named pilot batch**, validating each user across all three data types, managing **coexistence** during the split, then expanding **wave by wave**. Artifact names are fixed — create them with the exact names shown and reuse them every wave. Migration-**tool internals** (the data migration service job screens, GWMME, or a third-party tool) are deferred to the vendor's own docs; this guide governs the repeatable pattern around them.

> *Model: baseline and design once, prove the migration and cutover on a pilot batch, validate mail/calendar/contacts per user against the gate, then cut over DNS/MX and expand. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

Step 1 — Assess: baseline the source mailboxes

1. Run `GWGMAIL-Discovery.ps1` to baseline the Exchange Online source: total and licensed users,

shared mailboxes, resource/room mailboxes, and groups read from Microsoft 365 / Entra through Microsoft Graph (read-only). Record it as `GWGMAIL Discovery - Source Mailboxes - Baseline`.

1. Add the details the API cannot read: per-mailbox data volumes, contact counts, resource-calendar

mappings, and any mail-touching integrations (relay connectors, forwarding rules, shared-mailbox delegation) that affect the migration or routing.

1. Record gaps against the Requirements checklist. Do not proceed past missing source access, missing

target licensing, or a missing identity source of truth — resolve those first.

Step 2 — Configure: fix the tooling and data-types design

1. Choose the migration **tool/pattern** and record it in `GWGMAIL Design - Tooling & Data Types - Mail`:

• **Data migration service (Admin console)** — migrate email (and calendar) from Exchange Online

from the Admin console, using **delta migrations** to finalize recent mail at cutover; needs a Workspace super admin and a source-side Microsoft Entra administrator, and migrates up to the per-batch user limit Google states (**250 users at a time**) ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service), [Google Workspace Updates](https://workspaceupdates.googleblog.com/2025/06/migrate-email-calendar-content-from-microsoft-exchange-online-to-google-workspace.html)).

• **Desktop tool (GWMME)** — migrate **mail, calendar, and contacts** together for multiple users;

the pattern to choose when contacts must move in the same run ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme)).

• **Third-party tool** — a named tool whose internal steps are deferred to its vendor docs.

1. Fix which **data types** move and in what order (typically mail and calendar aligned to the mail

cutover, contacts alongside), and the **mappings** required — resource/room calendars and attendee/target-email mappings so calendar events resolve correctly ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration)).

1. Decide the **delta** approach: an initial bulk pass ahead of cutover, then a delta pass at cutover

that brings only new mail without duplicating.

1. Author the design **tool-agnostically** and **freeze** it before the pilot. Every wave runs this

same design.

Step 3 — Configure: design and stand up coexistence

1. Design the transition state as `GWGMAIL Coexistence - Mail Routing - Transition`:

• **Mail routing.** Choose **split delivery** — point the domain **MX at Google**, and have Gmail

route onward to Exchange Online the mail for users not yet migrated, per a routing rule keyed to the migrated population (the usual per-wave pattern) — or **dual delivery** where a copy goes to both systems ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery), [Google — dual delivery](https://support.google.com/a/answer/9228551)).

• **Calendar interop.** Define how free/busy resolves across the two platforms during the split so

cross-platform scheduling works.

• **Lower the MX TTL** (for example 300 seconds) ahead of the cutover so routing/DNS changes take

effect quickly and a back-out is fast.

1. Stand up the routing on a test host/domain and prove it: a test message reaches the correct

platform for a migrated and a not-yet-migrated user, and a cross-platform meeting resolves free/busy.

1. Set coexistence's intended **end** — it is transitional, decommissioned as the estate completes,

never permanent.

Step 4 — Configure: build the pilot batch

1. Assemble `GWGMAIL Pilot - Mail Batch - Wave 0`: a small, representative set (core IT and early

adopters, plus at least one shared mailbox and one resource calendar so those paths are exercised).

1. Build the batch's target group with `GWGMAIL-Set-ScopeTargeting.ps1` (runs with `-WhatIf` by

default; re-run with `-WhatIf:$false` to apply).

Step 5 — Author the DNS/MX cutover runbook and rollback

1. Author `GWGMAIL Cutover - DNS/MX - Per Wave` with the fixed sequence:

2. **Readiness** — the batch's target accounts are provisioned and licensed, the tool can read the source, and `GWGMAIL-Validation.ps1` reports the batch ready.

1. **Migrate mail/cal/contacts** — run the tool for the batch: bulk mail + calendar + contacts ahead of cutover.

1. **Coexistence routing** — apply the split/dual-delivery routing so the batch's mail follows them to Gmail, under change control.

1. **Validate** — run `GWGMAIL-Validation.ps1` per user (mail **and** calendar **and** contacts) and `GWGMAIL-Report.ps1` for coverage; run a **delta pass** to finalize recent mail.

1. **DNS/MX cutover** — apply the staged MX/DNS change for the wave with a documented back-out.

2. **Rollback** — revert routing/DNS to the prior platform and re-point the batch, used only if the gate fails.

1. Define the go/no-go: the wave passes only when readiness held, all three data types validated per user, the delta pass is clean, and the DNS/MX change is applied and reversible.

Step 6 — Pilot: run Wave 0 end to end

1. Build the Wave 0 group with `GWGMAIL-Set-ScopeTargeting.ps1 -WhatIf:$false`.

2. Execute the full runbook on the pilot batch: readiness → migrate mail/cal/contacts → coexistence routing → validate → DNS/MX cutover.

1. Run `GWGMAIL-Validation.ps1` per user and `GWGMAIL-Report.ps1` for coverage/exceptions. The pilot passes only when the whole runbook ran end to end and every user validated across all three data types.

Step 7 — Validate: fix the runbook, not just the batch

1. Complete the Testing Guide for the pilot. Treat every issue as a **runbook/design** defect: fix the tooling design, the mappings, or the coexistence routing so later waves inherit the fix.

1. Remediate coverage/data gaps with `GWGMAIL-Remediation.ps1` (runs with `-WhatIf` by default) — for example re-queuing a mailbox's failed items or normalizing the wave members list — then re-run validation and the delta pass.

Step 8 — Expand wave by wave

1. For each subsequent wave, re-run the same runbook: build the wave group

(`GWGMAIL-Set-ScopeTargeting.ps1`), migrate mail/cal/contacts, apply coexistence routing, validate per user (`GWGMAIL-Validation.ps1`), report coverage (`GWGMAIL-Report.ps1`), run the delta pass, and cut over DNS/MX — passing the go/no-go before the next wave.

1. Keep the coexistence routing rule current as the migrated population grows so every user's mail

always reaches their current platform; a stale rule silently mis-delivers.

Step 9 — Decommission coexistence

1. After the **final** wave validates and cuts over, retire the split/dual-delivery routing and the

onward route to Exchange Online, and finalize the MX so all mail is delivered in Gmail — never mid-migration. Confirm no forwarding loop or stale route remains.

Order of operations

```
Assess/baseline -> Design (tooling+data types) -> Coexistence -> Pilot batch -> Cutover runbook+rollback -> Wave 0 pilot -> Validate/fix runbook -> Expand wave by wave -> Decommission coexistence
```

5. Testing Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Run these validation gates for **every** batch — first for the pilot (`GWGMAIL Pilot - Mail Batch - Wave 0`), then for each expansion wave — before that wave's DNS/MX cutover and go/no-go. Each test confirms one part of the migration and cutover worked for the wave's population. A wave does not pass until its gate criteria are met, and a mailbox is not "migrated" until mail **and** calendar **and** contacts all validate.

Test 1 — Batch membership is complete and correct

Do: Confirm every user intended for the wave is in the wave's target group and no one is in two waves; run `GWGMAIL-Set-ScopeTargeting.ps1` (preview) to confirm membership. Expected: The batch's members match the plan; no duplicate or missing assignment.

Test 2 — Readiness: target accounts provisioned and licensed

Do: Run `GWGMAIL-Validation.ps1` for the batch. Expected: Every batch user resolves in the source and has a Gmail-bearing target account provisioned and licensed; readiness signals report present with no gaps.

Test 3 — Mail migrated

Do: For a sample of batch users, confirm mailbox contents (folders/labels, read/unread, history) landed in Gmail; run a **delta pass** and confirm it brings only new mail without duplicating ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-wit

h-the-new-data-migration-service)). Expected: Sampled users see their mail in Gmail; the delta pass adds only new items, no duplicates.

Test 4 — Calendar migrated

Do: For a sample of batch users, confirm primary and additional calendars, events (all-day, attendees, status, sensitivity, reminders), and any resource/room calendars imported per the mapping ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration)). Expected: Calendars and events resolve in Google Calendar with attendees and resource mappings intact.

Test 5 — Contacts migrated

Do: For a sample of batch users, confirm contacts landed in Google Contacts. Expected: Non-deleted contacts appear for the sampled users, matching the source.

Test 6 — Coexistence routes mail correctly

Do: Send a test message to a migrated batch user and to a not-yet-migrated user; send one from each back out. Expected: Each message reaches the correct platform per the split/dual-delivery design — no loss, no double delivery — matching `GWGMAIL Coexistence - Mail Routing - Transition` ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery)).

Test 7 — Cross-platform free/busy resolves

Do: Create a cross-platform meeting between a migrated user and a not-yet-migrated user. Expected: Free/busy resolves both ways and the invite is delivered on both platforms.

Test 8 — DNS/MX cutover applied under change control

Do: Confirm the wave's MX/DNS change was applied with an approved change record, a lowered MX TTL, and a documented back-out. Expected: The change is recorded, reversible, and mail follows the wave's migrated users to Gmail.

Test 9 — Coverage and exceptions reported

Do: Run `GWGMAIL-Report.ps1` for the batch. Expected: Per-user coverage across mail/calendar/contacts renders; exceptions (users missing a data type) are listed for remediation, not hidden.

Test 10 — Rollback is proven available

Do: Review the wave's rollback step and confirm the back-out (revert routing/DNS, re-point the batch) is documented and executable. Expected: A clear, tested rollback exists; it is invoked only if the gate fails.

Pass/fail gate

Gate	Requirement
Pilot (Wave 0) pass	Tests 1–10 pass; every pilot user validated across mail, calendar, and contacts; every runbook/design defect fixed before expanding
Expansion wave pass	Tests 1–10 pass for the wave; the DNS/MX cutover applied and reversible; go/no-go cleared before the next wave
Coexistence decommission	All waves migrated, validated, and cut over before the split/dual-delivery routing and onward route are retired

6. Troubleshooting Guide — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Use `GWGMAIL-Discovery.ps1` for the source baseline, `GWGMAIL-Validation.ps1` for per-user readiness and migration validation, `GWGMAIL-Report.ps1` for coverage/exceptions, and `GWGMAIL-Remediation.ps1` (runs with `-WhatIf` by default) for safe fixes to batch-readiness data gaps. Migration-**tool** failures (a specific job erroring inside the data migration service, GWMME, or a third-party tool) are diagnosed in the tool's own docs; this guide covers the **migration pattern**: data types, coexistence, cutover, and validation.

A. A user's mail migrated but calendar or contacts did not

Symptom	Cause	Remediation
Mail present in Gmail, calendar empty	Calendar not included in the run, or resource/attendee mapping missing	Confirm calendar is in scope in `GWGMAIL Design - Tooling & Data Types - Mail`, supply the resource/attendee mapping, and re-run calendar for the user ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration))
Contacts missing	Tool/path chosen does not move contacts(e.g. Admin-console mail/calendar path)	Use the desktop tool (GWMME) for contacts, or migrate contacts separately, per the design ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme))
Events import without rooms/attendees	Resource-calendar / attendee mapping not provided	Provide the mapping and re-run; validate with Testing Guide Test 4

B. Migrated mail appears duplicated

• A re-run duplicated items instead of adding only new mail. Use a **delta migration** for re-runs —

it brings only mail that arrived since the last pass without duplicating — rather than re-running a full migration ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service)). Re-validate with Testing Guide Test 3.

C. Mail is lost or double-delivered during coexistence

Symptom	Cause	Remediation
Migrated user's mail still lands in Exchange Online	Split-delivery routing not updated after the batch cut over	Update `GWGMAIL Coexistence - Mail Routing - Transition` so the batch's users deliver in Gmail; re-test with Testing Guide Test 6 ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery))
A message is delivered to both platforms	Dual delivery left on for users who have cut over	Switch those users to split-delivery routing (single target = Gmail); confirm no forwarding loop remains
Not-yet-migrated user's mail bounces	No onward route to Exchange Online	Confirm the onward host/route to Exchange Online exists per the coexistence design

D. Cross-platform free/busy fails

• A migrated and a not-yet-migrated user cannot see each other's availability. Confirm the calendar

interop in `GWGMAIL Coexistence - Mail Routing - Transition` is live for both directions and re-run Testing Guide Test 7 before continuing the wave.

E. The DNS/MX cutover did not take effect (or cannot be reversed quickly)

• Routing changes are slow to apply, or a back-out is stuck. The MX **TTL** was likely not lowered

before the cutover. Set a low MX TTL (for example 300 seconds) ahead of the cutover so changes and back-outs propagate fast; confirm the change was applied under change control with a documented back-out (Testing Guide Test 8).

F. A batch fails validation

Symptom	Cause	Remediation
Readiness fails	Target accounts not provisioned/licensed in Google	Provision/license the batch's Gmail accounts; re-run `GWGMAIL-Validation.ps1`
Coverage incomplete	A data type did not migrate for some users	Do not cut over; re-run the missing data type, run `GWGMAIL-Report.ps1`, remediate exceptions
Membership wrong	Duplicate or missing wave assignment	Rebuild the target group with `GWGMAIL-Set-ScopeTargeting.ps1`; reconcile with `GWGMAIL-Remediation.ps1`

G. The same problem recurs every wave

- A defect fixed only for one batch will reappear. Fix the **runbook**, the tooling/data-types design,

or the coexistence routing — not just the batch — so later waves inherit the fix (Implementation Step 7). Recurring issues are almost always an un-fixed design, not bad luck.

H. A cutover needs to be reversed

- The batch failed validation after the routing/DNS change. Invoke the runbook's rollback: revert the

coexistence routing/DNS change to the prior platform under change control and re-point the batch. Rollback is only used when the gate fails — confirm it was tested (Testing Guide Test 10).

I. Graph scripts fail to connect

- `GWGMAIL-Discovery.ps1`, `GWGMAIL-Validation.ps1`, `GWGMAIL-Report.ps1`, and

`GWGMAIL-Set-ScopeTargeting.ps1` use Microsoft Graph against the source M365/Entra tenant. If connection fails, confirm the Microsoft Graph PowerShell SDK is installed and the signed-in account holds the read scopes the script requests (consent on first run). Re-run after consent.

J. Diagnostics quick reference

```
.\GWGMAIL-Discovery.ps1                                          # source-mailbox baseline
.\GWGMAIL-Set-ScopeTargeting.ps1 -GroupName "..." -MembersCsv .\batch.csv   # -WhatIf by default
.\GWGMAIL-Validation.ps1 -Wave "Wave 0" -MembersCsv .\batch.csv -TargetProvisioned -CoexistenceLive
.\GWGMAIL-Report.ps1 -CoverageCsv .\gwgmail-coverage.csv          # per-user mail/cal/contacts coverage
.\GWGMAIL-Remediation.ps1 -MembersCsv .\batch.csv                # -WhatIf by default; add -WhatIf:$false t
```

K. When to escalate

If a batch's target accounts are provisioned and licensed, coexistence routing is correct, the tool reports the migration complete, and validation still reports gaps after remediation and a delta pass, capture the `GWGMAIL-Validation.ps1` and `GWGMAIL-Report.ps1` output and review the batch with the identity, mail-flow, and migration-tool owners before cutting over DNS/MX.

7. Best Practices — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Field-proven practices for moving mailboxes to Gmail **without losing data and without breaking mail flow** — focused on cutover and coexistence.

• **Pilot the whole thing first.** Prove the full migration and DNS/MX cutover on a named pilot batch

(`GWGMAIL Pilot - Mail Batch - Wave 0`) of core IT and early adopters — including at least one shared mailbox and one resource calendar — before any production wave. It is cheaper to find a mapping or routing defect in 50 users than in a business unit of thousands.

• **Validate all three data types per user.** A mailbox is not migrated until mail **and** calendar

and contacts validate. Declaring "migrated" on mail alone hides missing calendars and contacts until users complain ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration)).

• **Bulk-migrate ahead, delta at cutover.** Run an initial bulk pass before the cutover and a **delta

migration** at cutover so only new mail moves — no duplicates, no long freeze at go-live ([Google — migrate email with the data migration service](https://knowledge.workspace.google.com/admin/migrate/migrate-email-with-the-new-data-migration-service)).

• **Stand up coexistence before the first cutover, and time-box it.** Mail routing and calendar

interop must be live before anyone cuts over so migrated and not-yet-migrated users still mail and schedule each other — and coexistence is transitional, decommissioned as waves complete, never left running indefinitely ([Google — split delivery](https://knowledge.workspace.google.com/admin/gmail/advanced/send-email-to-2-email-systems-with-split-delivery), [Google — dual delivery](https://support.google.com/a/answer/9228551)).

• **Keep the routing rule following users.** As each wave completes, update the split/dual-delivery

rule so every user's mail always reaches their current platform; a stale rule silently mis-delivers or double-delivers.

• **Lower the MX TTL before the cutover.** Drop the MX TTL (for example to 300 seconds) ahead of the

cutover so routing/DNS changes apply quickly and a back-out is fast; raise it again once the estate is stable.

• **Treat the DNS/MX cutover as a change-controlled step, not a switch flip.** Apply every MX/DNS

change under change control with a documented, tested back-out; a high-impact mail-flow change without a rollback is not ready to run.

• **Get the calendar mappings right up front.** Resource/room calendars and attendee/target-email

mappings must be defined before the pilot, or events import without their rooms and attendees ([Google — what's imported from Exchange Online](https://knowledge.workspace.google.com/admin/migrate/whats-migrated-in-an-exchange-online-migration)).

- **Enable smart features ahead of the mail migration.** For the data migration service, enable smart features and personalization at least 24 hours before migrating so migrated mail sorts and searches correctly ([Google — about GWMME](https://knowledge.workspace.google.com/admin/migrate/about-gwmme)).
- **Fix the runbook, not just the batch.** Every issue found in a wave is a defect in the runbook, the tooling design, or the coexistence routing — fix it there so later waves inherit the fix, so each wave stops re-learning the same lesson.
- **Provision and license the target first, always.** A mailbox cannot land in an account that does not exist; provision each wave's Gmail accounts and licenses before any data moves or any routing addresses them.
- **Defer tool internals to the vendor.** This solution governs the pilot-first pattern, validation, coexistence, and cutover; the data migration service job, GWMME, and third-party tools' internal screens live in their own docs — keep the pattern tool-agnostic so a tooling change touches the tool layer, not the migration design.
- **Report coverage and exceptions cumulatively.** Track users migrated vs total and per-user mail/calendar/contacts coverage every cadence with `GWGMAIL-Report.ps1`; a stalled coverage trend or a growing exception list is an early warning.

8. Deliverables Summary — Exchange Online → Gmail Migration (Mail, Calendar, Contacts)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Exchange-to-Gmail migration model — data types, tooling patterns, coexistence/mail routing, DNS/MX cutover — and the deployment model (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: baseline, tooling & coexistence design, pilot batch, per-user validation, DNS/MX cutover, expand by wave
5. Testing Guide	Per-batch validation gates (mail/calendar/contacts, coexistence, cutover) with pass/fail criteria

Document	Purpose
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Cutover and coexistence without losing data

Named artifacts

Artifact	Name
Source mailbox/estate baseline	`GWGMAIL Discovery - Source Mailboxes - Baseline`
Tooling & data-types design	`GWGMAIL Design - Tooling & Data Types - Mail`
Coexistence / mail-routing design	`GWGMAIL Coexistence - Mail Routing - Transition`
Pilot migration batch	`GWGMAIL Pilot - Mail Batch - Wave 0`
Per-wave target group	`GWGMAIL Wave - Target Group - Per Wave`
Per-user validation record	`GWGMAIL Validation - Mail/Cal/Contacts - Per User`
DNS/MX cutover runbook	`GWGMAIL Cutover - DNS/MX - Per Wave`

PowerShell scripts

Script	What it does
`GWGMAIL-Discovery.ps1`	Baselines the Exchange Online / M365 source estate for mail migration: user/mailbox counts, licensed users, shared mailboxes, resource/room mailboxes, and groups (Microsoft Graph, read-only)
`GWGMAIL-Validation.ps1`	Per-user migration validation — confirms a batch's users resolve, are licensed, the target is provisioned, and mail/calendar/contacts and coexistence signals are asserted (read-only)
`GWGMAIL-Report.ps1`	Renders migration coverage and exceptions: users migrated vs total and per-user mail/calendar/contacts coverage (Microsoft Graph, read-only)
`GWGMAIL-Remediation.ps1`	Safe remediation of migration data gaps in a batch members list (duplicates, blanks, unresolved UPNs, failed-item re-queue list); runs with `-WhatIf` by default
`GWGMAIL-Set-ScopeTargeting.ps1`	Builds a wave's target group in Microsoft Entra from a members CSV; runs with `-WhatIf` by default

The mail-migration playbook (outline)

A single reusable playbook the migration runs from, assembled from the artifacts above:

1. **Source baseline** — `GWGMAIL Discovery - Source Mailboxes - Baseline`: mailbox/user counts,

data volumes, shared/resource mailboxes, contact counts, and mail-touching integrations.

1. **Tooling & data-types design** — `GWGMAIL Design - Tooling & Data Types - Mail`: the tool/pattern

(data migration service / GWMME / third-party), data types and order, mappings, and the delta plan.

1. **Coexistence design** — `GWGMAIL Coexistence - Mail Routing - Transition`: split/dual delivery,

calendar interop, lowered MX TTL, and the intended decommission point.

1. **Pilot batch** — `GWGMAIL Pilot - Mail Batch - Wave 0`: core IT + early adopters, a shared mailbox

and a resource calendar included.

1. **DNS/MX cutover runbook** — `GWGMAIL Cutover - DNS/MX - Per Wave`: readiness → migrate

mail/cal/contacts → coexistence routing → validate → DNS/MX cutover → rollback.

1. **Per-user validation** — `GWGMAIL Validation - Mail/Cal/Contacts - Per User`: mail **and**

calendar **and** contacts confirmed per user.

1. **Expansion & decommission** — expand wave by wave with the go/no-go gate, then retire coexistence

routing once the estate is fully on Gmail.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable mail migration following the model: Assess, then Configure, then Pilot, then Validate,

then Expand.

Outcome

Every in-scope mailbox on Gmail with **calendars and contacts intact** — coexistence managed during the transition, the DNS/MX cutover governed and reversible, and the whole migration proven on a pilot batch before any production wave — so nothing is lost and users land smoothly.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWGMAIL-Discovery.ps1

Purpose: Captures the source-mailbox baseline the migration is planned from: total and licensed user counts, resource/room mailboxes, and group/distribution-list counts read from the Microsoft 365 / Entra source through Microsoft Graph. Read-only — it supports the Assess step, moves no mail, and changes no mail flow. Per-mailbox data volumes, contact counts, and mail-touching integrations the API cannot read are recorded by hand into the baseline artifact. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Discovery.ps1 Optional: -CsvPath .\gwgmail-baseline.csv to export the summary. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: User.Read.All, Organization.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total users, licensed users, resource/room mailboxes, and group count, with a PASS/GAP line per signal you record against the Requirements checklist as GWGMAIL Discovery - Source Mailboxes - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Discovery.ps1 Optional: -CsvPath .\gwgmail-baseline.csv to export the summary. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: User.Read.All, Organization.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total users, licensed users, resource/room mailboxes, and group count, with a PASS/GAP line per signal you record against the Requirements checklist as GWGMAIL Discovery - Source Mailboxes - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

GWGMAIL-Remediation.ps1

Purpose: Cleans common data-quality gaps that block a batch from passing validation, working only on a local members/coverage CSV — never on tenant data, mail flow, or money. It flags duplicate UPNs, blank UPN rows, and (when a source is reachable) UPNs that do not resolve in the Microsoft 365 source, and it builds a re-queue list of users still missing a data type from an optional coverage CSV. When applied it writes a normalized, de-duplicated members CSV. Runs with -WhatIf by default, so it only reports unless you explicitly turn -WhatIf off. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Remediation.ps1 `-MembersCsv .\batch.csv Apply: ...\GWGMAIL-Remediation.ps1 -MembersCsv .\batch.csv` -OutputPath .\batch.clean.csv -CoverageCsv .\gwgmail-coverage.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the source tenant (needs the Microsoft Graph SDK and User.Read.All). -CoverageCsv (columns Upn, Mail, Calendar, Contacts) produces a re-queue list of users missing a data type. Without -Resolve, only local de-duplication and blank checks run. EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and the re-queue list, with what would change. When applied: a cleaned members CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [string]$CoverageCsv, [switch]$Resolve)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Remediation.ps1 `-MembersCsv .\batch.csv Apply: ...\GWGMAIL-Remediation.ps1 -MembersCsv`

`.\batch.csv` -OutputPath .\batch.clean.csv -CoverageCsv .\gwgmail-coverage.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the source tenant (needs the Microsoft Graph SDK and User.Read.All). -CoverageCsv (columns Upn, Mail, Calendar, Contacts) produces a re-queue list of users missing a data type. Without -Resolve, only local de-duplication and blank checks run. EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and the re-queue list, with what would change. When applied: a cleaned members CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [string]$CoverageCsv, [switch]$Resolve)

GWGMAIL-Report.ps1

Purpose: Produces the standing coverage report: per-user mail, calendar, and contacts coverage and the exception list (users missing a data type), plus overall coverage against the source population. It reads a per-user coverage CSV (one row per user with Mail/Calendar/Contacts status) and cross-checks the total population against the Microsoft 365 source through Microsoft Graph. Read-only against Graph — it modifies nothing and moves no mail. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Report.ps1 ` -CoverageCsv .\gwgmail-coverage.csv -CsvPath .\gwgmail-report.csv The coverage CSV should include columns: Upn, Mail, Calendar, Contacts (each Yes/No/Pending). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the total source-user count for the coverage denominator). EXPECTED RESULTS A per-data-type coverage summary, an exception list of users missing a data type, an overall coverage percentage vs the source total, and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CoverageCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Report.ps1 ` -CoverageCsv .\gwgmail-coverage.csv -CsvPath .\gwgmail-report.csv The coverage CSV should include columns: Upn, Mail, Calendar, Contacts (each Yes/No/Pending). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the total source-user count for the coverage denominator). EXPECTED RESULTS A per-data-type coverage summary, an exception list of users missing a data type, an overall coverage percentage vs the source total, and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$CoverageCsv, [string]$CsvPath)

GWGMAIL-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) the Microsoft Entra security group that defines a migration wave — the pilot batch (Wave 0) first, then each expansion wave — so every wave runs against a bounded, well-understood population and coexistence routing can key off it. Optionally seeds members from a CSV of user UPNs. This is the only included script that writes to the tenant: it creates a group and adds members, nothing else — no licenses changed, no mail flow altered, no mail moved. Runs with -WhatIf by default. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Set-ScopeTargeting.ps1 `-GroupName "GWGMAIL Wave - Wave 0 - Pilot"` `-MembersCsv .\batch.csv` Apply: `...\GWGMAIL-Set-ScopeTargeting.ps1` -GroupName "GWGMAIL Wave - Wave 0 - Pilot" -MembersCsv .\batch.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and

User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook and coexistence routing. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Set-ScopeTargeting.ps1 `-GroupName "GWGMAIL Wave - Wave 0 - Pilot" -MembersCsv .\batch.csv Apply: ...\GWGMAIL-Set-ScopeTargeting.ps1 -GroupName "GWGMAIL Wave - Wave 0 - Pilot" -MembersCsv .\batch.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook and coexistence routing. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

GWGMAIL-Validation.ps1

Purpose: Validates that a migration batch is ready and that each user's data types are accounted for before a DNS/MX cutover: the batch's users (from a members CSV) resolve in the Microsoft 365 source and are licensed, and the target-provisioned, mail, calendar, contacts, and coexistence signals are asserted per the runbook. This is the per-user validation gate. Read-only against Microsoft Graph — it changes nothing and moves no mail. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Validation.ps1 -Wave "Wave 0" ` -MembersCsv .\batch.csv -TargetProvisioned -MailMigrated -CalendarMigrated -ContactsMigrated -CoexistenceLive -MembersCsv is a CSV with a 'Upn' column listing the batch's users. The switches assert the target account is provisioned and each data type migrated; -CoexistenceLive asserts mail routing/calendar interop is live. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "BATCH READY" when every listed user resolves and is licensed and the target, mail, calendar, contacts, and coexistence signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$TargetProvisioned, [switch]$MailMigrated, [switch]$CalendarMigrated, [switch]$ContactsMigrated, [switch]$CoexistenceLive)

How to run: powershell -ExecutionPolicy Bypass -File .\GWGMAIL-Validation.ps1 -Wave "Wave 0" ` -MembersCsv .\batch.csv -TargetProvisioned -MailMigrated -CalendarMigrated -ContactsMigrated -CoexistenceLive -MembersCsv is a CSV with a 'Upn' column listing the batch's users. The switches assert the target account is provisioned and each data type migrated; -CoexistenceLive asserts mail routing/calendar interop is live. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "BATCH READY" when every listed user resolves and is licensed and the target, mail, calendar, contacts, and coexistence signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$TargetProvisioned, [switch]$MailMigrated, [switch]$CalendarMigrated, [switch]$ContactsMigrated, [switch]$CoexistenceLive)